

QCar2 Internship

Quang Thinh BUI

April 2026

Flatness-Based Bézier Trajectory Generation with Single Track Model for QCar2

1 Introduction

This report presents a flatness-based trajectory generation approach for the kinematic single-track vehicle model. The formulation is inspired by the flatness-based obstacle avoidance MPC framework proposed in [1] while adopting a trajectory generation methodology similar to the Bézier-based formulation used for QCar2.

The Cartesian position of the vehicle is selected as the flat output. Using differential flatness, all vehicle states and control inputs can be reconstructed from the flat output and its derivatives. A Bézier polynomial parameterization is then employed to generate smooth trajectories satisfying boundary conditions up to the second derivative.

2 Vehicle Model

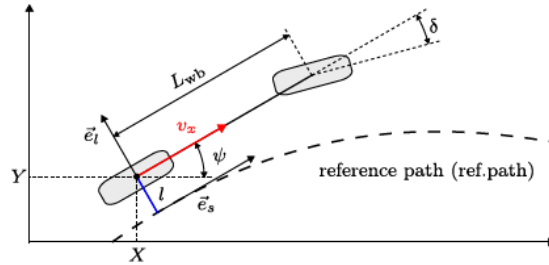


Fig. 1. Kinematic single-track model.

Figure 1: Kinematic single-track model.

The vehicle dynamics are modeled using the kinematic single-track model

$$\dot{X} = v_x \cos \psi, \quad \dot{Y} = v_x \sin \psi, \quad (1)$$

$$\dot{\psi} = \frac{v_x}{L_{wb}} \tan \delta, \quad \dot{v}_x = a_x, \quad (2)$$

where (X, Y) denote the Cartesian coordinates of the vehicle, ψ is the heading angle, v_x is the longitudinal velocity, δ is the steering angle, a_x is the longitudinal acceleration, and L_{wb} is the wheelbase.

The state and input vectors are defined as

$$x = [X \quad Y \quad \psi \quad v_x]^T, \quad u = [\delta \quad a_x]^T. \quad (3)$$

3 Differential Flatness

3.1 Flat Output Selection

The flat output is chosen as the Cartesian position of the vehicle:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}. \quad (4)$$

Its first derivative is

$$\dot{z} = \begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \end{bmatrix} = \begin{bmatrix} v_x \cos \psi \\ v_x \sin \psi \end{bmatrix}. \quad (5)$$

Therefore, the longitudinal velocity and heading angle can be recovered as

$$v_x = \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \quad \psi = \arctan\left(\frac{\dot{z}_2}{\dot{z}_1}\right). \quad (6)$$

Differentiating the heading angle gives

$$\dot{\psi} = \frac{\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1}{\dot{z}_1^2 + \dot{z}_2^2}. \quad (7)$$

Using the vehicle model relation

$$\dot{\psi} = \frac{v_x}{L_{wb}} \tan \delta, \quad (8)$$

the steering angle becomes

$$\delta = \arctan\left(\frac{L_{wb}(\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1)}{(\dot{z}_1^2 + \dot{z}_2^2)^{3/2}}\right). \quad (9)$$

The longitudinal acceleration is recovered from

$$a_x = \frac{\dot{z}_1 \ddot{z}_1 + \dot{z}_2 \ddot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \quad (10)$$

Consequently, all states and inputs can be expressed as functions of the flat output and its derivatives:

$$X = z_1, \quad Y = z_2, \quad (11)$$

$$\psi = \arctan\left(\frac{\dot{z}_2}{\dot{z}_1}\right), \quad v_x = \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \quad (12)$$

$$\delta = \arctan\left(\frac{L_{wb}(\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1)}{(\dot{z}_1^2 + \dot{z}_2^2)^{3/2}}\right), \quad (13)$$

$$a_x = \frac{\dot{z}_1 \ddot{z}_1 + \dot{z}_2 \ddot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \quad (14)$$

3.2 Boundary Conditions

At a given time instant, let

$$x = [X \quad Y \quad \psi \quad v_x]^T, \quad u = [\delta \quad a_x]^T. \quad (15)$$

The flat output and its derivatives are written as

$$z = \begin{bmatrix} X \\ Y \end{bmatrix}, \quad (16)$$

$$\dot{z} = \begin{bmatrix} v_x \cos \psi \\ v_x \sin \psi \end{bmatrix}, \quad (17)$$

$$\ddot{z} = \begin{bmatrix} a_x \cos \psi - \frac{v_x^2}{L_{yb}} \tan \delta \sin \psi \\ a_x \sin \psi + \frac{v_x^2}{L_{wb}} \tan \delta \cos \psi \end{bmatrix}. \quad (18)$$

Thus, each waypoint is converted into a complete flat boundary condition:

$$(X_i, Y_i, \psi_i, v_{x,i}, \delta_i, a_{x,i}) \longrightarrow (z_i, \dot{z}_i, \ddot{z}_i). \quad (19)$$

4 Trajectory Generating

4.1 Flatness-based Bézier Trajectory Generating

The trajectory is generated from the flat output

$$z(t) = \begin{bmatrix} z_1(t) \\ z_2(t) \end{bmatrix} = \begin{bmatrix} X(t) \\ Y(t) \end{bmatrix}. \quad (20)$$

Boundary conditions are imposed up to the second derivative:

$$z(0), \dot{z}(0), \ddot{z}(0), \quad z(T), \dot{z}(T), \ddot{z}(T). \quad (21)$$

Since the highest derivative order is $r = 2$, the Bézier polynomial degree is chosen as

$$n + 1 = 2(r + 1) = 6 \quad \Rightarrow \quad n = 5. \quad (22)$$

The degree-5 Bézier polynomial is defined as

$$z(s) = \sum_{i=0}^5 P_i B_i^5(s), \quad s \in [0, 1], \quad (23)$$

where

$$B_i^5(s) = \binom{5}{i} (1-s)^{5-i} s^i. \quad (24)$$

The normalized time variable is introduced as

$$s = \frac{t}{T}. \quad (25)$$

The first derivative is

$$\dot{z}(s) = \frac{5}{T} \sum_{i=0}^4 (P_{i+1} - P_i) B_i^4(s). \quad (26)$$

The second derivative is

$$\ddot{z}(s) = \frac{20}{T^2} \sum_{i=0}^3 (P_{i+2} - 2P_{i+1} + P_i) B_i^3(s). \quad (27)$$

The Bézier control points are obtained directly from the boundary conditions.

At $s = 0$:

$$P_0 = z(0), \quad (28)$$

$$P_1 = z(0) + \frac{T}{5} \dot{z}(0), \quad (29)$$

$$P_2 = z(0) + \frac{2T}{5} \dot{z}(0) + \frac{T^2}{20} \ddot{z}(0). \quad (30)$$

At $s = 1$:

$$P_5 = z(T), \quad (31)$$

$$P_4 = z(T) - \frac{T}{5}\dot{z}(T), \quad (32)$$

$$P_3 = z(T) - \frac{2T}{5}\dot{z}(T) + \frac{T^2}{20}\ddot{z}(T). \quad (33)$$

Therefore, each trajectory segment can be completely determined from the initial and final flat boundary conditions.

4.2 Automatic Computation of Boundary Conditions from Waypoint Geometry

In practice, the user does not specify all vehicle states and inputs at the waypoints. Instead, the user specifies the Cartesian position of all waypoints, the heading angle at the first and last waypoints, and the travelling time of each segment.

For a sequence of N waypoints, the Cartesian position of waypoint i is defined as

$$p_i = \begin{bmatrix} X_i \\ Y_i \end{bmatrix}, \quad i = 0, \dots, N-1. \quad (34)$$

The prescribed boundary headings are

$$\psi_0, \quad \psi_{N-1}, \quad (35)$$

and the segment travelling times are

$$T_i, \quad i = 0, \dots, N-2, \quad (36)$$

where T_i is the travelling time from waypoint i to waypoint $i+1$.

The remaining quantities required by the flatness-based trajectory generation, namely the intermediate heading angles, the longitudinal velocities, the steering angles, and the longitudinal accelerations, are automatically computed from the waypoint geometry and the segment times.

Thus, the initial user-defined data

$$p_i, \quad \psi_0, \quad \psi_{N-1}, \quad T_i \quad (37)$$

are converted into complete boundary waypoints of the form

$$q_i = [X_i \quad Y_i \quad \psi_i \quad v_{x,i} \quad \delta_i \quad a_{x,i}]^T. \quad (38)$$

These complete boundary waypoints are then mapped into the flat output boundary conditions

$$z_i, \quad \dot{z}_i, \quad \ddot{z}_i, \quad (39)$$

which are required to construct the degree-5 Bézier trajectory.

Heading angle computation

The heading angle at the first and last waypoints is directly imposed by the user:

$$\psi_0 = \psi_{\text{start}}, \quad \psi_{N-1} = \psi_{\text{goal}}. \quad (40)$$

The intermediate heading angles are computed from the local geometry of the waypoint sequence.

For an intermediate waypoint i , the normalized incoming and outgoing direction vectors are

$$d_i^- = \frac{p_i - p_{i-1}}{\|p_i - p_{i-1}\|}, \quad d_i^+ = \frac{p_{i+1} - p_i}{\|p_{i+1} - p_i\|}. \quad (41)$$

The local tangent direction is approximated by the sum of the incoming and outgoing directions:

$$\tilde{h}_i = d_i^- + d_i^+. \quad (42)$$

The corresponding unit heading direction is obtained by normalization:

$$\hat{h}_i = \frac{\tilde{h}_i}{\|\tilde{h}_i\|} = \begin{bmatrix} \cos \psi_i \\ \sin \psi_i \end{bmatrix}, \quad i = 1, \dots, N-2. \quad (43)$$

Therefore, the intermediate heading angle is computed as

$$\psi_i = \text{atan2}(\hat{h}_{i,y}, \hat{h}_{i,x}), \quad i = 1, \dots, N-2. \quad (44)$$

The function atan2 is used in order to preserve the correct quadrant of the heading angle.

Path curvature and steering angle computation

The steering angle is computed from the local path curvature. From the kinematic single-track model, the heading dynamics are

$$\dot{\psi} = \frac{v_x}{L_{wb}} \tan \delta. \quad (45)$$

For a planar path, the curvature is defined as the variation of the heading angle with respect to the traveled distance:

$$\kappa = \frac{d\psi}{ds}. \quad (46)$$

Since the traveled distance satisfies

$$\frac{ds}{dt} = v_x, \quad (47)$$

The curvature can also be written as

$$\kappa = \frac{d\psi/dt}{ds/dt} = \frac{\dot{\psi}}{v_x}. \quad (48)$$

Substituting the heading dynamics into this expression gives

$$\kappa = \frac{1}{v_x} \left(\frac{v_x}{L_{wb}} \tan \delta \right) = \frac{\tan \delta}{L_{wb}}. \quad (49)$$

Therefore, if the path curvature is known, the steering angle can be computed as

$$\delta = \arctan(L_{wb}\kappa). \quad (50)$$

However, at this stage, the curvature cannot be computed directly from $\kappa = \dot{\psi}/v_x$. Although the heading angles are known at the waypoints, they are only discrete boundary orientations. They do not yet define a continuous heading profile $\psi(t)$ along the whole path.

For this reason, the curvature used for the steering-angle computation is estimated directly from the geometry of three consecutive waypoints p_{i-1} , p_i , and p_{i+1} . This provides a purely geometric approximation of the local path bending before the continuous Bézier trajectory is constructed.

The three vectors involved in the curvature computation are

$$p_i - p_{i-1} = \begin{bmatrix} X_i - X_{i-1} \\ Y_i - Y_{i-1} \end{bmatrix}, \quad (51)$$

$$p_{i+1} - p_i = \begin{bmatrix} X_{i+1} - X_i \\ Y_{i+1} - Y_i \end{bmatrix}, \quad (52)$$

and

$$p_{i+1} - p_{i-1} = \begin{bmatrix} X_{i+1} - X_{i-1} \\ Y_{i+1} - Y_{i-1} \end{bmatrix}. \quad (53)$$

The signed curvature at an intermediate waypoint is approximated as

$$\kappa_i = \frac{2[(X_i - X_{i-1})(Y_{i+1} - Y_i) - (Y_i - Y_{i-1})(X_{i+1} - X_i)]}{\sqrt{(X_i - X_{i-1})^2 + (Y_i - Y_{i-1})^2} \sqrt{(X_{i+1} - X_i)^2 + (Y_{i+1} - Y_i)^2} \sqrt{(X_{i+1} - X_{i-1})^2 + (Y_{i+1} - Y_{i-1})^2}}. \quad (54)$$

The numerator corresponds to the signed two-dimensional cross product between two consecutive path segments. Hence, its sign indicates the turning direction of the path. A positive value corresponds to a left turn, while a negative value corresponds to a right turn. If the three points are aligned, the numerator is zero and the curvature is also zero.

At the first and last waypoints, the curvature is set to zero:

$$\kappa_0 = 0, \quad \kappa_{N-1} = 0. \quad (55)$$

Finally, the steering angle at each waypoint is obtained from the geometric curvature as

$$\delta_i = \arctan(L_{wb}\kappa_i). \quad (56)$$

Longitudinal velocity computation

The longitudinal velocity is computed from the segment times. The length of segment i , connecting waypoint i to waypoint $i + 1$, is

$$d_i = \|p_{i+1} - p_i\|, \quad i = 0, \dots, N - 2. \quad (57)$$

Since the segment time T_i is prescribed, the average velocity along this segment is

$$\bar{v}_{x,i} = \frac{d_i}{T_i}. \quad (58)$$

However, the flatness-based boundary conditions require a longitudinal velocity at each waypoint. Therefore, the waypoint velocities are obtained from the adjacent segment velocities.

At the first waypoint,

$$v_{x,0} = \bar{v}_{x,0}. \quad (59)$$

At the last waypoint,

$$v_{x,N-1} = \bar{v}_{x,N-2}. \quad (60)$$

For an intermediate waypoint, the velocity is chosen as the average of the two neighboring segment velocities:

$$v_{x,i} = \frac{\bar{v}_{x,i-1} + \bar{v}_{x,i}}{2}, \quad i = 1, \dots, N - 2. \quad (61)$$

Longitudinal acceleration computation

For the kinematic single-track model used in this work, the longitudinal acceleration a_x is an input of the system. Therefore, after the waypoint velocities have been computed, the longitudinal acceleration is approximated from the variation of velocity over the segment times.

For segment i , the average longitudinal acceleration is

$$\bar{a}_{x,i} = \frac{v_{x,i+1} - v_{x,i}}{T_i}, \quad i = 0, \dots, N - 2. \quad (62)$$

The acceleration at the first waypoint is taken from the first segment:

$$a_{x,0} = \bar{a}_{x,0}. \quad (63)$$

The acceleration at the last waypoint is taken from the last segment:

$$a_{x,N-1} = \bar{a}_{x,N-2}. \quad (64)$$

For an intermediate waypoint, a centered approximation is used:

$$a_{x,i} = \frac{\bar{a}_{x,i-1} + \bar{a}_{x,i}}{2}, \quad i = 1, \dots, N-2. \quad (65)$$

Thus, the automatically computed acceleration is consistent with the variation of the longitudinal velocity along the waypoint sequence.

Construction of flat boundary conditions

After computing ψ_i , $v_{x,i}$, δ_i , and $a_{x,i}$, each waypoint is written as a complete vehicle boundary condition:

$$q_i = [X_i \ Y_i \ \psi_i \ v_{x,i} \ \delta_i \ a_{x,i}]^T. \quad (66)$$

Using the flatness relations, the corresponding flat output is

$$z_i = \begin{bmatrix} X_i \\ Y_i \end{bmatrix}. \quad (67)$$

The first derivative of the flat output is

$$\dot{z}_i = \begin{bmatrix} v_{x,i} \cos \psi_i \\ v_{x,i} \sin \psi_i \end{bmatrix}. \quad (68)$$

The second derivative of the flat output is

$$\ddot{z}_i = \begin{bmatrix} a_{x,i} \cos \psi_i - \frac{v_{x,i}^2}{L_{wb}} \tan \delta_i \sin \psi_i \\ a_{x,i} \sin \psi_i + \frac{v_{x,i}^2}{L_{wb}} \tan \delta_i \cos \psi_i \end{bmatrix}. \quad (69)$$

Thus, the initial user-defined data

$$p_i, \quad \psi_0, \quad \psi_{N-1}, \quad T_i \quad (70)$$

are converted into complete flat boundary conditions of the form

$$(z_i, \dot{z}_i, \ddot{z}_i). \quad (71)$$

For the segment connecting waypoint i to waypoint $i+1$, the initial boundary conditions are

$$z(0) = z_i, \quad \dot{z}(0) = \dot{z}_i, \quad \ddot{z}(0) = \ddot{z}_i, \quad (72)$$

and the final boundary conditions are

$$z(T_i) = z_{i+1}, \quad \dot{z}(T_i) = \dot{z}_{i+1}, \quad \ddot{z}(T_i) = \ddot{z}_{i+1}. \quad (73)$$

These boundary conditions are then used to compute the control points of the degree-5 Bézier curve for each segment.

Finally, the generated trajectory should satisfy the physical constraints of the QCar2, such as

$$|\delta(t)| \leq \delta_{\max}, \quad |a_x(t)| \leq a_{x,\max}. \quad (74)$$

If these constraints are violated, the waypoint geometry may require a turn that is too sharp, or the prescribed segment time may be too short. In that case, the waypoints should be smoothed, the segment time should be increased, or the generated velocity profile should be reduced.

References

- [1] A. L. Gratzner, M. M. Broger, A. Schirrer, and S. Jakubek, “Flatness-Based Mixed-Integer Obstacle Avoidance MPC for Collision-Safe Automated Urban Driving,” *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*, 2023.